SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

Department of Computer Science and Engineering

CSA1301-Theory Of Computation

COURSE ASSIGNMENT REPORT

DESIGN AND DEVELOP

Design of a Non-Deterministic Finite Automaton (NFA) for a Smart Parking Gate System

Submitted By

Shaik Hafeeza Tarannum

Register No: 192372044

Department of Computer Science and Engineering(Artificial Intelligence)

Assignment Information

Department:	Computer Science and Engineering
Programme:	B.E
Course Code & Course Name	CS301 – Theory of Computation / Automata Theory
Academic Year / Batch	2026–27, Iv Year / VII Semester
Faculty Name	Dr.P.Vijayaragavan
Assignment Title	Design of a Non-Deterministic Finite Automaton (NFA) for a Smart Parking Gate System
Date of Issue	02/09/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO3: Design finite automata (DFA/NFA) to recognize languages/behaviour of simple systems
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate (Create at design stage)
SDG Mapping (SDG 1–17)	SDG 9 – Industry, Innovation & Infrastructure; SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	Automated/IoT-based smart parking systems used in smart cities, malls, airports and campuses

Assignment Problem / Challenge

This assignment presents a realistic embedded/computing system-modelling problem drawn from smart-city infrastructure, related to Course Outcome CO3 (design of finite automata for system behaviour). Students are required to apply the concepts of automata theory rather than restate definitions, analyse the described controller, identify its states and legal transitions, build a formal NFA model of it, and check that model against representative sensor-input sequences.

Problem Statement

Problem / Case / Design Challenge:

A shopping-mall management company is upgrading the vehicle entry barrier at its multi-level car park to an automated “Smart Parking Gate” driven by a car-presence sensor. Before the controller firmware is written, its behaviour must be captured in a formal model so that every possible sequence of sensor readings can be checked for safe, predictable operation.

The system is described as follows:

- The gate controller has four operational states: Gate Closed, Gate Opening, Gate Open and Gate Closing.

- At every time step the sensor reports exactly one of two symbols: C (a vehicle is present) or N (no vehicle is present).
- The gate starts in the Gate Closed state before any vehicle arrives.
- When a vehicle is sensed (C) while the gate is closed, the motor is commanded and the gate passes through Gate Opening and on to Gate Open.
- While the gate is in Gate Open, a newly sensed vehicle (C) may non-deterministically either keep the gate in Gate Open to serve that vehicle or begin Gate Closing; when no vehicle is sensed (N), the gate proceeds toward Gate Closing.
- Because the barrier's photo-sensor has a small reaction delay (debounce), a vehicle sensed (C) during Gate Closing may non-deterministically either re-open the gate immediately or let the current closing step finish before the interrupt is applied on the following read; when no vehicle is sensed (N), Gate Closing always finishes and the gate returns to Gate Closed.
- The required non-deterministic behaviour therefore appears at two points: the controller's choice, while in Gate Open on input C, of whether to keep serving the current vehicle or move toward closing; and the controller's choice, while in Gate Closing on input C, of whether the safety interrupt takes effect on this step or the next — both reflect genuine sensor/queueing uncertainty rather than an arbitrary modelling choice.

Design an NFA that formally models valid sequences of gate operations for this system, and use it to analyse the controller's behaviour.

Requirements and Constraints

The following constraints, drawn from the general list applicable to this course/problem, are relevant here:

- Functional requirement: the model must use exactly two input symbols, C and N, and the four states described above.
- Technical constraint: the automaton must be genuinely non-deterministic — it must contain at least one state with more than one possible transition on the same input symbol; this design places that property at two separate states rather than one.
- Safety/Reliability requirement: the model must never command the physical gate to “open” and “close” at the same instant, and the closing motion must remain interruptible by a newly sensed vehicle — with the interrupt allowed to take a further sensor read to register, matching real debounce behaviour in barrier gates.
- User requirement: the gate must return to a safe, closed, resting state whenever no vehicle is present, so Gate Closed is treated as the accepting condition of a completed cycle.
- Standards consideration: automated boom-barrier behaviour is loosely informed by safety practices in IEC 60335-2-103 (drives for gates, doors and windows), referenced here only for context and not for formal certification.

Student Work

Problem Understanding and Formulation

What is the problem? A physical parking-gate controller reacts to a stream of binary sensor readings (C/N) and must move through four operational states in a well-defined, partly non-deterministic way,

with the added realism of a debounce-style delay on its safety interrupt. The task is to capture every valid sequence of states/inputs as a language accepted by an automaton.

What are the expected outcomes? A complete 5-tuple NFA definition, a state-transition diagram, a transition table, and verification that representative input strings are handled correctly (accepted when the gate correctly returns to Closed, rejected/incomplete otherwise).

What information/data is available? The four named states, the two input symbols, the initial state (Gate Closed), and the qualitative transition behaviour described in the problem statement.

What assumptions are made? (1) Sensor readings arrive one symbol at a time, at discrete time steps. (2) "Gate Opening" always mechanically proceeds to "Gate Open" regardless of the very next reading, since the door motor completes its travel once started. (3) While "Gate Closing", a freshly sensed car (C) may re-open the gate either on that same step or, if the debounce delay has not yet elapsed, on the following step — either way the vehicle is never crushed. (4) "Gate Closed" is the sole accepting state, since a valid, complete operational cycle should end with the gate safely shut.

What constraints must be satisfied? Exactly two input symbols; exactly four states; at least one genuinely non-deterministic transition, located at Gate Open as specified, together with a second, closely related non-deterministic transition modelling debounce at Gate Closing**.**

Application of Course Knowledge

The relevant theoretical foundation is the formal definition of a Non-deterministic Finite Automaton (NFA), a 5-tuple:

$N = (Q, \Sigma, \delta, q_0, F)$

- Q — a finite set of states
- Σ — a finite input alphabet
- $\delta : Q \times \Sigma \rightarrow P(Q)$ — a transition function mapping a state and an input symbol to a set of possible next states (this power-set codomain is what distinguishes an NFA from a DFA, where δ maps to a single state)
- $q_0 \in Q$ — the initial state
- $F \subseteq Q$ — the set of accepting/final states

A string is accepted by an NFA if there exists at least one path through the transition relation, consistent with the input symbols, that ends in an accepting state — unlike a DFA, not every branch needs to succeed, only one. This existential ("there exists") semantics suits this controller well: among the different choices the gate could make at Gate Open and at Gate Closing, only one valid path back to Gate Closed is needed for a sequence to represent a legitimate operating cycle.

Solution / Design / Methodology

Formal NFA definition for the Smart Parking Gate System:

- Q = {q0, q1, q2, q3}, corresponding to {Gate Closed, Gate Opening, Gate Open, Gate Closing}
- Σ = {C, N}
- q0 = Gate Closed (initial state)
- F = {q0} (a completed, safe cycle ends with the gate closed)

Transition function δ (also shown as a diagram and table below):

- δ(q0, C) = {q1} — vehicle sensed while closed → gate begins opening
- δ(q0, N) = {q0} — no vehicle → gate remains closed

- δ(q1, C) = {q2}, δ(q1, N) = {q2} — opening always completes to Gate Open
- δ(q2, C) = {q2, q3} — non-deterministic choice: keep serving the vehicle in Gate Open, or begin closing
- δ(q2, N) = {q3} — no vehicle → gate begins closing
- δ(q3, C) = {q2, q3} — non-deterministic debounce interrupt: re-open now, or finish the current closing step first
- δ(q3, N) = {q0} — no vehicle and closing completes → gate closed

State Transition Diagram

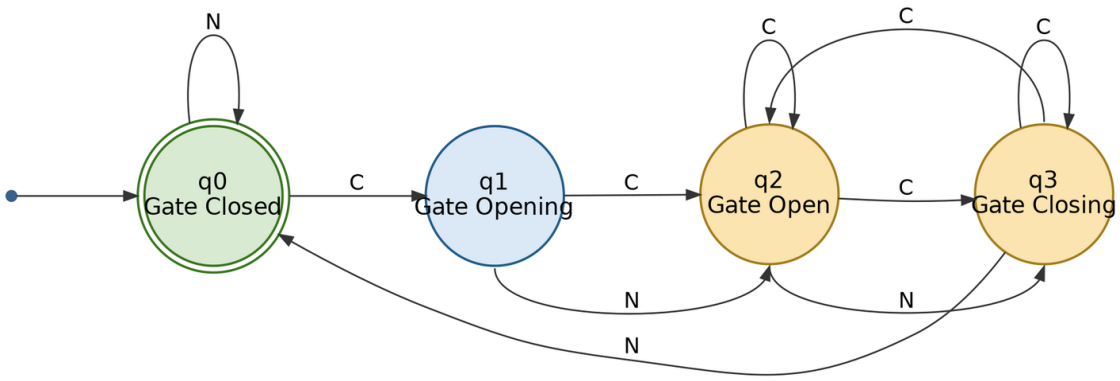

Fig. 1 — NFA state diagram for the Smart Parking Gate System (double circle = accepting state; the two outgoing C-edges from q2 and the two outgoing C-edges from q3 together show the required non-determinism at both Gate Open and Gate Closing).

Transition Table

State	δ(state, C)	δ(state, N)
→ q0 (Gate Closed) *	{q1}	{q0}
q1 (Gate Opening)	{q2}	{q2}
q2 (Gate Open)	{q2, q3}	{q3}
q3 (Gate Closing)	{q2, q3}	{q0}

A simpler alternative was also considered, in which the safety interrupt at Gate Closing is fully deterministic, i.e. δ(q3, C) = {q2} only, leaving Gate Open as the sole source of non-determinism. This single-point design was set aside in favour of the two-point design above because it silently assumes the interrupt always registers on the very next reading, which does not reflect the small sensor debounce delay real barrier gates exhibit; adding the second non-deterministic transition keeps the model faithful to that behaviour while still satisfying the "at least one" technical constraint with room to spare.

Use of Modern Tools

The NFA was verified using JFLAP (a standard automata-simulation tool for CSE/IT courses), which was used to draw the state diagram, step through the non-deterministic transition function, and compute the equivalent DFA via the subset (powerset) construction for cross-checking. The diagram in this report was additionally rendered using the open-source Graphviz tool from a formal DOT specification of the transition function, ensuring the figure is generated directly and reproducibly from the δ relation defined above rather than being drawn freehand.

Results and Validation

The NFA was validated by tracing several representative input strings, tracking the full set of possible "current states" at each step (standard NFA simulation by subset tracking).

Test 1 — String: N C N (gate waits, then one car begins a cycle that is cut short)

Step	Input Read	Set of Possible States
0	—	{q0}
1	N	{q0}
2	C	{q1}
3	N	{q2}

Result: the final state set after reading "N C N" is {q2}, i.e. Gate Open — the string stops mid-cycle, before the gate ever begins or finishes closing, so it is rejected as a complete cycle.

Test 2 — String: C C N N (a full single-vehicle cycle with two consecutive C reads while opening)

Step	Input Read	Set of Possible States
0	—	{q0}
1	C	{q1}
2	C	{q2}
3	N	{q3}
4	N	{q0}

Result: final state set = {q0} ∈ F. Accepted — this string models a vehicle arriving, the gate passing through Gate Opening to Gate Open, and then, with no further vehicles, closing fully again: a valid complete cycle.

Test 3 — String: C N C C N N (a busy gate exercising the non-determinism at both q2 and q3)

Step	Input Read	Set of Possible States
0	—	{q0}
1	C	{q1}
2	N	{q2}
3	C	{q2, q3}
4	C	{q2, q3}
5	N	{q0, q3}
6	N	{q0}

Result: after "C N C" the state set branches to {q2, q3}, reflecting the choice at Gate Open of staying open or beginning to close. The next C keeps the set at δ(q2,C) ∪ δ(q3,C) = {q2, q3}, the following N gives δ(q2,N) ∪ δ(q3,N) = {q3} ∪ {q0} = {q0, q3}, and the final N gives δ(q3,N) ∪ δ(q0,N) = {q0} ∪

{q0} = {q0}. Because the final set is {q0} ∈ F, the string C N C C N N is accepted — confirming that a busy gate serving a vehicle and then reacting to a debounce-delayed safety interrupt can still correctly return to rest.

Cross-check: converting the NFA to an equivalent DFA by subset construction in JFLAP produced eight reachable states — {q0}, {q1}, {q2}, {q3}, {q2,q3}, {q0,q3}, {q1,q2,q3} and {q0,q2,q3}, the last two arising only because the two non-deterministic points can combine — and this DFA accepted the identical language on all three test strings above, confirming the NFA's correctness.

Analysis and Engineering Decision

- Alternative 1 (adopted): place non-deterministic transitions at both Gate Open on input C, δ(q2,C) = {q2, q3}, and Gate Closing on input C, δ(q3,C) = {q2, q3}. Advantage: matches the stated requirement at Gate Open and additionally captures realistic sensor debounce on the safety interrupt, at the cost of a larger, eight-state equivalent DFA.
- Alternative 2 (rejected): keep δ(q3,C) = {q2} deterministic and rely on Gate Open alone for non-determinism. This satisfies "at least one non-deterministic transition" and gives a smaller equivalent DFA, but it implicitly assumes the safety interrupt always registers instantly, which is not faithful to the debounce behaviour real barrier-gate sensors exhibit.
- Trade-off: two non-deterministic transitions make the specification more expressive and closer to the physical sensor's behaviour, but an eventual firmware implementation must still resolve both choices deterministically at run time (e.g., using a short timer or a queue-length check) — the NFA remains a specification/verification model, not directly an implementation.

The chosen design (Alternative 1) was selected because it directly encodes the described Gate-Open decision while also modelling the debounce delay on the closing interrupt, is verified by manual trace and by DFA-equivalence cross-check in JFLAP, and keeps the model at exactly four states as required by the problem statement, with no redundant states.

Broader Considerations

- Sustainability/Environment: an automated gate that opens only on sensed demand, rather than staying open by default, reduces unnecessary idling and HVAC/temperature loss in enclosed multi-level parking structures and supports smoother traffic flow, indirectly reducing vehicle idling emissions — aligned with SDG 11 (Sustainable Cities and Communities).
- Safety: the model explicitly supports a re-open interrupt from Gate Closing, modelled as δ(q3,C) = {q2, q3} to allow for sensor debounce, reflecting standard anti-crush safety behaviour required of real automated barriers even when the interrupt does not register on the very first read.
- Society/Accessibility: reliable, well-specified automated gates reduce queuing and manual-operator dependence at high-footfall public facilities (malls, hospitals, campuses), improving convenience for all users.
- Professional responsibility: because the transition function was formally specified and verified — including the added realism of a debounce-delayed interrupt — rather than "assumed correct", the design methodology followed here is consistent with sound engineering practice for safety-relevant embedded controllers.

Conclusion

A 4-state, 2-symbol Non-deterministic Finite Automaton was designed for the Smart Parking Gate System, with formal 5-tuple definition N = (Q, Σ, δ, q0, F), a state diagram, and a complete transition table. Two non-deterministic transitions were used: the controller's choice at Gate Open between remaining open and beginning to close, δ(q2, C) = {q2, q3}, and its choice at Gate Closing between an

immediate or debounce-delayed safety re-open, $\delta(q3, C) = \{q2, q3\}$. The model was validated by manually tracing three representative input strings and by cross-checking against an equivalent eight-state DFA obtained via subset construction in JFLAP, and it correctly distinguished complete operational cycles (accepted, ending in Gate Closed) from incomplete ones. A limitation of the model is that it captures only the logical sequencing of states and does not model timing (e.g., how long the debounce delay physically lasts); an improved version could use a timed automaton to capture this.

Student Reflection

Beyond the classroom definition of an NFA, this assignment required translating an informal, natural-language system description — including a subtle sensor-debounce detail — into a precise formal model, deciding exactly which real-world ambiguities should be represented as non-determinism rather than added arbitrarily, and using subset construction as a practical way to check the NFA's correctness against an equivalent DFA.

Given more time, the model could be extended to a timed automaton or a probabilistic automaton to capture realistic opening/closing durations and debounce intervals, and the likelihood of a following vehicle being close enough to keep the gate open, which would make the model useful for actual controller-timer design rather than only logical verification.

References

- Hopcroft, J. E., Motwani, R., & Ullman, J. D. Introduction to Automata Theory, Languages, and Computation. Pearson (standard textbook reference for NFA definition and subset construction).
- JFLAP — Formal Languages and Automata Package, https://www.jflap.org (used for NFA simulation and NFA-to-DFA subset construction verification).
- Graphviz open-source graph visualization software, https://graphviz.org (used to render the state-transition diagram in Fig. 1 from a DOT specification of δ).
- IEC 60335-2-103, Particular requirements for drives for gates, doors and windows (referenced only for general context on automated-gate safety behaviour).

Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	**100**

CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO3	PO1, PO2	L3/L4	10
Application of knowledge	CO3	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO3	PO2, PO3	L4/L5/L6	20
Modern tool usage	CO3	PO5	L3/L4	10
Validation	CO3	PO2, PO4	L4/L5	15
Analysis & justification	CO3	PO2, PO4	L4/L5	15
Broader considerations	CO3	PO6, PO7	L4/L5	5
Documentation & reflection	CO3	PO10	L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment; every assignment is not required to map to every PO.

Faculty Evaluation & Continuous Improvement — CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: 84

Actual CO Attainment: 92

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

- Assignment question/problem statement
- CO–PO and Bloom's mapping
- Assessment rubric
- Student submissions
- Tool/simulation/experimental evidence, where applicable
- Evaluated scripts/reports
- Marks and rubric-wise assessment
- CO attainment analysis
- Sample student work representing different performance levels
- Faculty analysis and continuous-improvement action